# Regularization

ICT 3.3 Machine learning and Data Mining

Emanuela Boros, May 2026

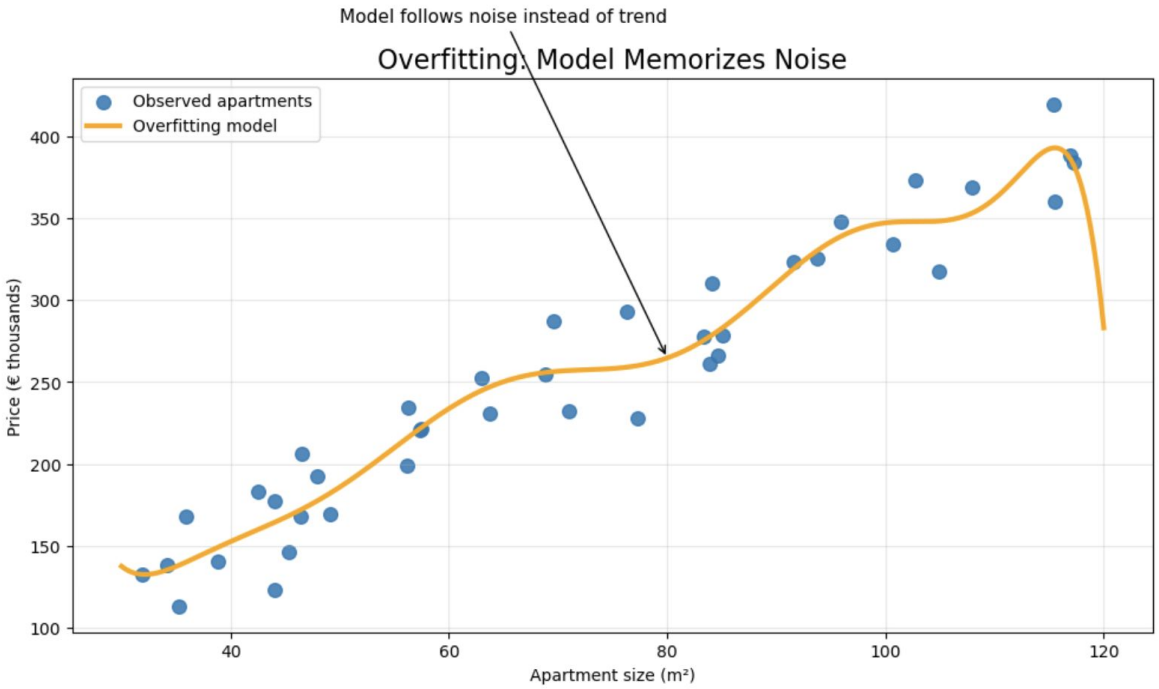
# ML Challenges

# ML Challenges

- underfit → too simple
- overfit → too complex
- good fit → generalizes

# Overfitting

Connecting all points memorizes the data, but does not capture the pattern.



# Overfitting – Why this happens?

1 Model too complex

- too many parameters
- very flexible (e.g., high-degree polynomial)

→ fits *every point*, including noise

2 Not enough data

- small dataset
- model memorizes instead of generalizing

3 Data is noisy or too complex

- too many features
- errors in labels
- random variation

→ model learns randomness

4 Training too long

- model keeps adapting to small fluctuations

5 No regularization

- nothing prevents large weights
- model becomes unstable

# How do we see a model is too complex?

dev loss goes down at first then starts going UP

Training improves, but **generalization** gets **worse**

# How do we prevent this?

# Regularization

- Prefer simpler models
- Avoid extreme behavior
- Balance fit vs simplicity

Regularization tells the model: *don't try too hard*. *Fit the data, but do not become too complicated.*

# Logistic Regression

Linear function:

$$z = wx + b$$

$$p(y = 1 \mid x) = \sigma(z)$$

Logistic regression:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Loss: $\mathcal{L} = -\sum [y \log p + (1 - y) \log(1 - p)]$

We train by minimizing the cross-entropy loss that says  if prediction is wrong → loss is large

A model adjusts w to reduce prediction error

If x ≈ 5:

- small weight:
  z = **0.5** · 5 = 2.5 → p ≈ 0.92
- large weight:
  z = **3** · 5 = 15→ p ≈ 1.0

→ larger w makes the sigmoid steeper → more sensitive predictions

If the data is separable, the loss keeps decreasing as w \to \infty

- small change in x
  → large change in z
  → huge jump in probability

# Regularization

- We already minimize error
- Now we add a penalty

→ We don't just care about fitting the data… we also care about keeping the model simple.

$$Loss = Error + \lambda \cdot Complexity$$

# What is “Complexity”?

- Large weights → complex model
- Small weights → smoother model

$$z = w_0 + w_1 x + w_2 x^2 + w_3 x^3 + \cdots + w_d x^d$$

→ Regularization pushes weights toward 0 (to make them smaller)

# Types of Regularization

L2 Regularization (Ridge)

- Penalizes large weights
- Keeps all features
- Smooth model
- Makes weights small

$$\text{Loss} = \text{Error} + \lambda \sum w_i^2$$

L1 Regularization (Lasso)

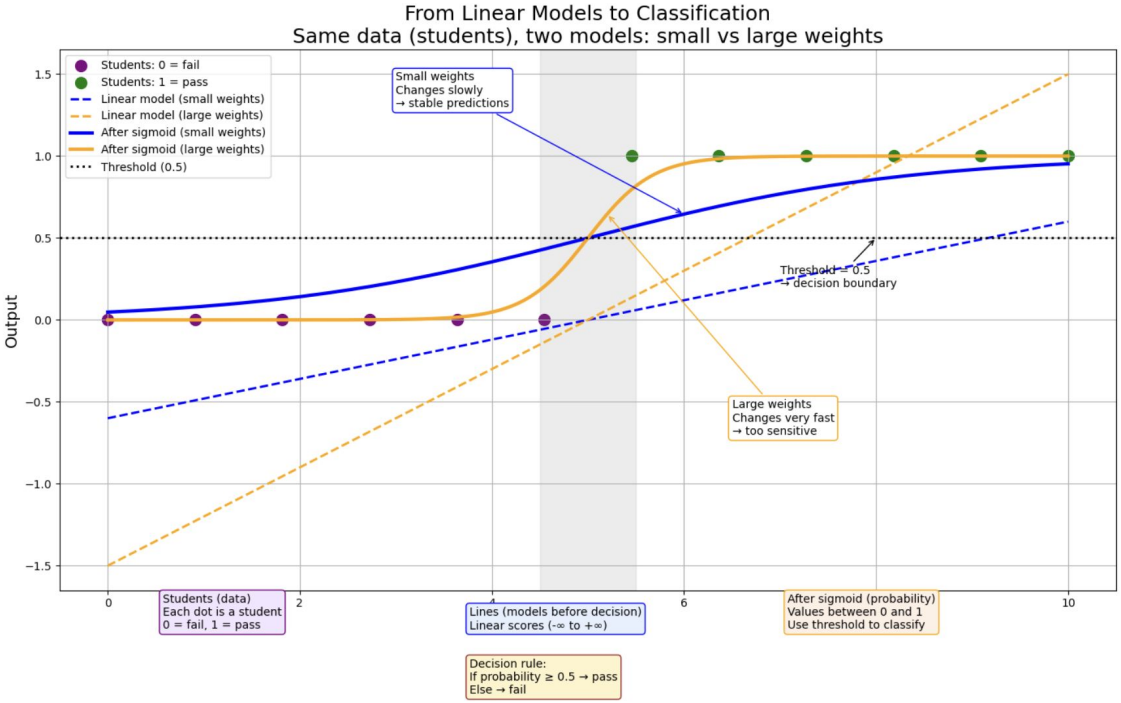
- Pushes some weights to 0
- Performs feature selection
- Can make some weights disappear

$$\text{Loss} = \text{Error} + \lambda \sum |w_i|$$

# Regularization

→ Blue model: both get similar predictions

→ Orange model: one fails, one passes instantly



# L2 Regularization (Ridge)

- punish large weights

large weights → unstable model
small weights → smoother model

→ pull weights toward zero, but not exactly zero

$$\text{Loss} = \text{Error} + \lambda \sum w_i^2$$

# L1 Regularization (Lasso)

- Pushes some weights exactly to zero

$$\text{Loss} = \text{Error} + \lambda \sum |w_i|$$

→ automatic feature selection

# Regularization - Lambda

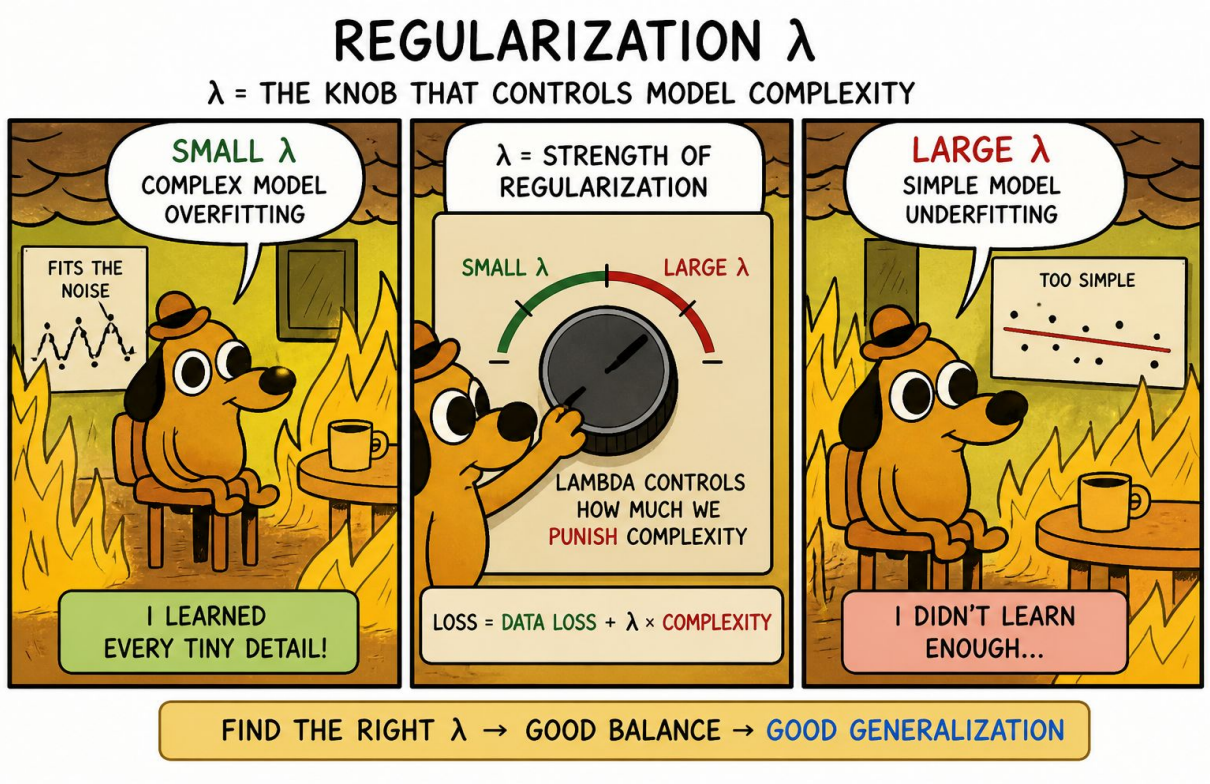
$\lambda$ = strength of regularization → the knob that controls model complexity.

- small $\lambda$ → complex model → overfitting
- large $\lambda$ → simple model → underfitting

→ Lambda controls how much we punish complexity.

# Regularization - Lambda

What happens if λ is too big?



# Mini Assignment

TODOs:

- Exercise notebook

Clone the repo!